BridgeHook: A Zero-Install Webhook Bridge Using a Browser Extension as the Forwarding Agent

Halleluyah Oludele
College of Medicine, University of Ibadan, Ibadan, Nigeria
halleluyaholudeele@gmail.com

June 2026

Abstract

Testing webhooks against a service running on localhost normally requires a public ingress: developers install a reverse-tunnel daemon (ngrok, cloudflared, localtunnel) or run a relay client (smee-client, gosmee) that subscribes to a hosted endpoint and re-POSTs payloads locally. Both add an install step and a long-lived local process. BridgeHook removes the install: the forwarding agent is a browser extension the developer already has a browser to run. The relay+SSE-to-localhost pattern itself is not new — smee.io, gosmee, and similar tools predate this work and are credited throughout. BridgeHook's contribution is a composition: a zero-install webhook bridge that uses a browser extension as the forwarding agent (escaping the CORS / Private-Network-Access wall that blocks page-JavaScript relays), backed by a Cloudflare Durable Object rendezvous, with per-event cryptographic request signing (an untrusted-relay model) and a hybrid push/poll delivery scheme resilient to the Manifest V3 service-worker lifecycle. We describe the architecture, trace every mechanism to the shipping implementation, give a qualitative feature comparison against existing tools, and specify a reproducible latency methodology. The tool runs today on Cloudflare Workers free-tier hosts; a custom-domain deployment is designed but not yet provisioned. We state the limitations plainly: Chromium/MV3 only, single-region relay, and dependence on the relay operator (mitigated, not eliminated, by signing).

1 Introduction

A webhook is an HTTP request a third-party service (Stripe, GitHub, a payment processor) sends to a developer-configured public URL when an event occurs [16, 4]. During development the handler under test runs on `localhost`, which has no public address. The standard remedy is to publish localhost to the internet, then point the webhook source at the published URL.

Two families of tools do this. *Reverse tunnels* (ngrok, cloudflared, localtunnel, Pinggy) run a local agent that opens an outbound connection to a cloud endpoint and exposes a public hostname bound to a local port [10, 2, 7, 11]. *Relay clients* (smee-client, gosmee) subscribe over Server-Sent Events (SSE) [19] to a hosted channel and re-issue each received payload as a local HTTP request [12, 1]. Each approach asks the developer to install something — a binary, an npm package, an SSH invocation — and to keep a local process alive for the duration of the session.

This paper describes BridgeHook, which keeps the relay+SSE design but moves the forwarding agent into a browser extension. A developer who has a Chromium browser open — which a web developer does — adds the extension once and needs no per-project install, no daemon, and no terminal. The thesis is narrow: the install friction of local webhook testing is removable by reusing the browser as the always-present forwarding host, provided the agent runs as an extension rather

than as page JavaScript. Section 2 explains why page JavaScript cannot do this and an extension can.

This paper makes the following contributions:

1. **Extension-as-agent forwarding.** We identify the browser extension as the one client form that can legally forward an HTTPS-relayed webhook to an `http://localhost` target, and show why a page-JavaScript relay (an approach that already exists [8]) cannot (Section 2).
2. **A Durable Object rendezvous and a hybrid push/poll delivery scheme** resilient to the Manifest V3 service-worker lifecycle (Sections 3.1, 3.3, 3.4).
3. **An untrusted-relay signing model**: per-event ECDSA signing of privileged client calls so a malicious or compromised relay cannot forge them, with the guarantee's exact scope and limits stated (Section 3.5).
4. **A qualitative comparison** against eight existing tools and a reproducible latency methodology (Section 5), with the implementation released as open source (Artifact Availability).

The relay+SSE transport itself is inherited from prior tools and is not claimed as novel; the contribution is the composition above.

2 Background and Browser Constraints

Webhooks. A producer POSTs an event body and headers (often an HMAC signature) to a configured URL and expects a timely HTTP response; the response status is used for retry decisions [16, 4]. A local-testing tool must therefore both deliver the request to localhost and return the local handler's response to the producer.

Reverse tunnels. ngrok, cloudflared, localtunnel, and Pinggy establish an outbound connection from a local agent and bind a public hostname to a local port [10, 2, 7, 11]. Pinggy reuses the system SSH client, avoiding a bespoke binary, but still requires a running SSH session and (on the free tier) reissues the URL after a timeout.

Relay plus SSE. smee.io exposes a public channel URL; the smee-client connects to it over SSE and re-POSTs payloads to a local target [13, 12]. gosmee reimplements both halves in Go and lets operators self-host the relay [1]. svix and Hookdeck provide production webhook infrastructure with CLIs whose `listen` subcommand forwards to localhost [17, 6]; Webhook Relay offers the same forward-to-localhost capability through a CLI or container agent [18]. BridgeHook's relay and SSE transport are this same pattern. What differs is the client: an extension, not a local process.

The browser security wall. If the forwarding agent were ordinary page JavaScript served over HTTPS, three browser policies would block it from reaching `http://localhost`:

- **Mixed content:** an HTTPS page may not issue a plain-HTTP subresource request, so `fetch("http://localhost:3000/...")` from an HTTPS dashboard is blocked or silently upgraded [9].
- **CORS:** a cross-origin response without permissive `Access-Control-Allow-*` headers is not readable by page script; a local dev server rarely sets these for an arbitrary cloud origin.

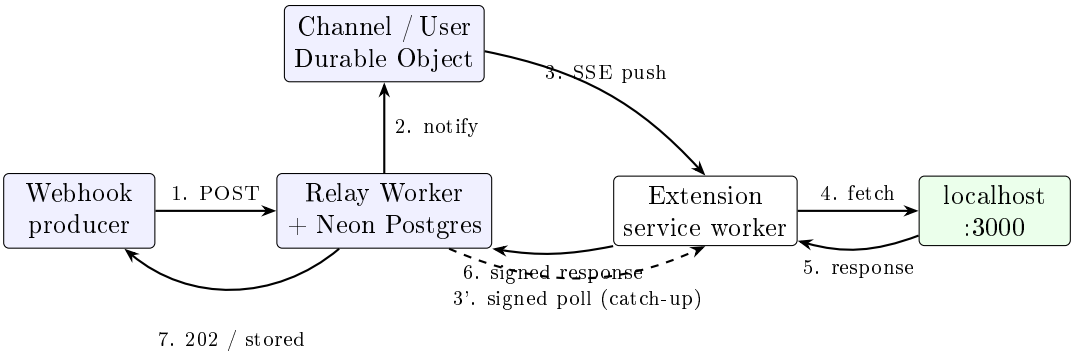

Figure 1: End-to-end dataflow. The producer's POST is persisted and fanned out to the extension over SSE (push) with signed polling as catch-up; the extension forwards to localhost and returns the captured response to the relay, signed.

- **Private Network Access (PNA):** the WICG draft (formerly CORS-RFC1918) restricts requests originating in a public context that target a private/local endpoint; Chrome sends a CORS preflight (`Access-Control-Request-Private-Network: true`) the local server must explicitly answer [15, 14].

A browser *extension* with an explicit `host_permissions` grant for `http://localhost/*` is not subject to these page-context restrictions for its own `fetch` calls; the user authorizes the local reach at install time. This is the single capability that lets the extension forward an HTTPS-relayed request to an HTTP localhost target, and it is why the agent must be an extension rather than a tab. A tab-based relay of exactly this shape exists: hooks-localhost broadcasts received webhooks over WebSocket to an open page, which then `fetch`es localhost [8]. It works only when the local dev server returns permissive CORS headers to the relay's origin, and only while the tab stays open. Moving the agent into an extension removes both constraints — the CORS-allowlist dependency and the kept-open tab. (Section 6 returns to the standing risk that PNA-style rules may eventually constrain extensions too.)

3 Design

Figure 1 shows the end-to-end path. The relay never contacts localhost; the extension is the only component that does.

3.1 Relay: Worker plus Durable Object rendezvous

The relay is a Cloudflare Worker built on Hono, backed by Neon PostgreSQL via Drizzle. Inbound webhooks are accepted either at a path (`/hook/:channelId`) or, when a tunnel apex domain is configured, at a wildcard subdomain (`<channelId>.<apex>`). Intake validates the channel, enforces a body-size cap and a path allowlist, applies the per-user daily event cap, persists an `events` row, and returns `202` immediately — it never blocks on localhost. Two tables back the system: `channels` (id, ECDSA public key, port, allowed paths, owner) and `events` (request method/path/headers/body, response status/headers/body, latency, claim fields).

Live delivery uses Cloudflare Durable Objects [3]. A Durable Object (DO) is a single addressable instance per id, so every SSE connection and every webhook notification for one channel reach the *same* instance and can share in-memory state. The `ChannelDO` holds open SSE writers in a `Set`;

on intake the Worker POSTs a notify to the DO, which fans the event out to all writers with `Promise.allSettled` and evicts any writer whose write rejects (a disconnected client). A per-channel connection cap rejects excess subscribers with `429`. A second DO type (`UserDO`) fans every event for any channel a signed-in user owns to a single cross-channel stream, which is what the extension subscribes to in push mode.

Authentication on the relay is gated by a single secret: when `BETTER_AUTH_SECRET` is set the relay runs in hosted mode with Better-Auth sessions and device tokens; when unset it runs in self-host mode with an implicit user and no auth routes. To stay within the Workers free-tier 10 ms CPU budget, password hashing uses a hand-rolled PBKDF2 rather than a heavier KDF.

3.2 Extension bridge: the forwarding agent

The extension is Manifest V3 with a module service worker, the permissions `storage`, `notifications`, `alarms`, and `host_permissions` for `http://localhost/*` plus the relay hosts. The localhost grant is the capability discussed in Section 2; with it the service worker's `fetch` to `http://localhost:PORT` is exempt from the page-context CORS and mixed-content rules that block a tab.

For each event the worker reconstructs the local request, strips infrastructure headers that must not be replayed (`host`, `cf-*`, `x-forwarded-*`, `content-length`, `transfer-encoding`, and similar), issues the `fetch` to localhost, measures latency with `performance.now()`, captures status, headers, and body, and returns that response to the relay. A connection refusal is reported to the user as a desktop notification rather than silently dropped.

3.3 Hybrid push/poll delivery

Delivery uses two paths feeding one handler. In *push* mode (available when the user is signed in via the dashboard session cookie) the worker holds a streaming-fetch SSE connection to `/api/me/stream`; the relay pushes a `webhook` frame the instant the event lands, and forwarding starts within milliseconds. In *poll* mode a per-service loop calls the signed events endpoint as catch-up and as the sole path when push is unavailable (self-host, device-token auth, or a dropped stream). A `Set` of handled event ids is shared across both paths, so an event delivered by push is filtered out of the next poll.

The poll cadence is adaptive: 2 s by default; 15 s while the SSE stream is healthy (polling is then only a safety net); 10 s after more than three consecutive errors; and 30 s when the relay reports the daily quota is exhausted (`402` with `code:"quota"`). The SSE reader reconnects with exponential backoff capped at 30 s.

3.4 MV3 service-worker resilience

A Manifest V3 background service worker is event-driven and is terminated on inactivity; any in-flight `setTimeout` loop dies with it [5]. A naive polling loop would therefore stop whenever Chrome reclaims the worker. BridgeHook treats the worker as disposable and rebuilds state on each wake (Figure 2). A `chrome.alarms` heartbeat fires every 30 s — the MV3 minimum period — and, on `onInstalled`, `onStartup`, and every cold boot, the worker *rehydrates*: it reloads services from `chrome.storage`, refreshes account state, and re-attaches a polling loop for any active service whose in-tick loop died. The user SSE stream is restarted if it died with the worker. Signing keys live in IndexedDB and survive worker death; a service whose key is missing is skipped rather than run unsigned.

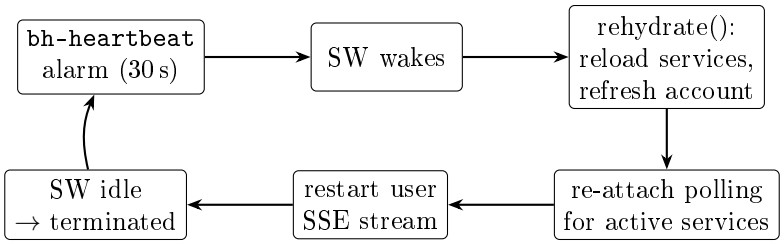

Figure 2: MV3 resilience loop. The worker is assumed to die between alarm ticks; each wake rebuilds the polling loops and SSE stream from durable storage rather than relying on in-memory continuity.

Listing 1: Canonical string signed per request (client and relay derive it identically).

```
canonical = METHOD + "\n"
          + pathname + "\n"
          + timestamp + "\n"
          + sha256_hex(body)
// headers: X-BH-Timestamp, X-BH-Signature (ECDSA P-256 / SHA-256)
```

3.5 Security model

The relay is treated as untrusted for integrity. Each channel owns a *non-extractable* ECDSA P-256 keypair generated in the browser via Web Crypto and stored in IndexedDB; only the raw public key is sent to the relay at channel creation. Every privileged client call (fetch events, post response, claim) is signed: the client builds a canonical string over method, path, timestamp, and a SHA-256 of the body, signs it, and attaches `X-BH-Timestamp` and `X-BH-Signature` (Listing 1). The relay re-derives the same string and verifies against the stored public key within a 60 s clock-skew window. Because the private key never leaves the browser and is non-extractable, a relay operator (or anyone who hijacks the SSE stream) can observe and reorder frames but cannot *forge* a signed claim or response. The scheme provides integrity and authenticity, not full anti-replay: the canonical string carries a timestamp but no nonce, and the relay keeps no used-signature cache, so a captured signed request can be replayed within the 60 s window. This is benign for the operations that are signed — the response upload is idempotent and the claim is a single-winner atomic update (control 3 below) — but it is not a general replay guarantee. The signature covers the path but not the query string, so query parameters (such as a result `limit`) are unauthenticated; no privileged decision depends on them.

Three further controls round out the model. (1) A per-channel *path allowlist* is enforced server-side at intake: a webhook whose post-prefix path is not in the channel's allowed set is rejected with `403`, so a leaked channel id cannot be used to drive arbitrary local routes (an empty allowlist permits any path, but the extension always sets a non-empty list at channel creation). (2) Authentication is three-tier in priority order: a dashboard session cookie (`SameSite=None`, sent with `credentials: "include"`, requiring no pairing); a long-lived device-token bearer (`dvc_...`) minted through an OAuth-style device-pairing flow for headless or signed-out use; and anonymous, permitted only against self-host relays. (3) Multiple executors on one channel (extension, dashboard tab, desktop app) arbitrate via an atomic claim — `UPDATE ... WHERE claimed_by_device_id IS NULL` — so exactly one forwards each event.

Table 1: Cloudflare free-tier allowances used by the relay deployment (from the project's deployment notes). Figures are platform allowances, not measured load.

Service	Free-tier allowance	Hosts
Cloudflare Pages	unlimited sites/bandwidth, 500 builds/mo	landing + dashboard
Cloudflare Workers	100,000 requests/day, 10 ms CPU/req	relay server
Durable Objects	1M requests/mo, 1 GB storage	per-channel + per-user state
Custom domain	~$12/year (not yet provisioned)	`*.bridgehook.dev` (design)

3.6 Stable URLs and port detection

A webhook URL pasted into Stripe must remain valid across browser restarts and worker deaths. When a signed-in user re-adds a service for a port that already has a channel, the extension reuses the existing channel id rather than minting a new one. If the local private key is gone (cleared storage, fresh profile), the extension generates a new keypair and re-keys the existing channel via a rotate-key call, keeping the same id and URL with fresh signing material. To reduce setup friction, the extension can HEAD-probe a set of common development ports (3000, 3001, 4000, 5000, 5173, 8000, 8080, 8888) and suggest the live ones.

4 Implementation

BridgeHook is a pnpm monorepo. `packages/shared` holds TypeScript types, constants, and the Drizzle schema; `relay/` is the Cloudflare Worker (Hono + Neon + Better-Auth); `apps/web` is the Vite/React landing page and dashboard; `apps/extension` is the MV3 extension; and `apps/desktop` is a planned Tauri tray app that would run the same bridge loop in Rust for users who want forwarding without a browser open. The DO-based relay and the ECDSA-signed client protocol are shared by the web dashboard, the extension, and (by design) the desktop bridge.

Deployment topology. The intended layout maps three properties to three subdomains of an apex (`bridgehook.dev` for docs, `app.bridgehook.dev` for the dashboard, `relay.bridgehook.dev` for the Worker). **That apex is not yet registered.** The tool currently runs on Cloudflare-provided hosts (`*.workers.dev` for the relay, `*.pages.dev` for the web app); the custom-domain layout is the deployment design, not a live claim. Everything fits Cloudflare's free tier (Table 1); the relay is stateless per request and DOs hold channel state in memory, so there is no separate database cost beyond Neon's free tier for durable event history.

5 Evaluation

5.1 Feature comparison

The qualitative comparison in Table 2 is factual and reflects each tool's documented behavior [10, 2, 7, 11, 12, 1, 17, 6]. The column where BridgeHook is alone is *install required*: every other tool ships a binary, an npm package, or an SSH session that must run locally, whereas BridgeHook's forwarding agent is a browser extension and runs no separate local process. Per-event request signing toward the relay is the second distinguishing column.

Table 2: Qualitative comparison of local webhook delivery tools. "Local process" = a daemon/CLI/SSH session the developer must keep running; "signing" = per-event cryptographic signing of the client→relay calls; "delivery" is the transport model: a held TCP *tunnel*, server-pushed *SSE*, or a *hybrid* of SSE push with polling fallback.

Tool	No install	No local proc.	No account	Self-host	Per-event sign	Delivery	Free
ngrok	no	no	no	no	n/a	tunnel	freemium
cloudflared	no	no	yes	no	n/a	tunnel	yes
localtunnel	no	no	yes	yes	n/a	tunnel	yes
Pinggy	yes*	no	yes	no	n/a	tunnel	freemium
smee-client	no	no	yes	yes	no	SSE	yes
gosmee	no	no	yes	yes	no	SSE	yes
svix-cli	no	no	yes	yes	no	SSE	freemium
Hookdeck CLI	no	no	no	no	no	SSE	freemium
BridgeHook	**yes**	**yes**	yes†	yes	**yes**	hybrid	yes

*Pinggy needs no binary but requires a running SSH session. †Anonymous use is supported only against a self-hosted relay; the hosted relay requires an account.

5.2 Latency methodology

We have not yet run the latency benchmark; this subsection specifies the reproducible methodology and the result tables are left with explicit placeholders to be filled from real runs. **No latency or throughput number in this paper is measured; all are marked "—".**

Setup. A local echo server records server-receive time; a load generator POSTs N events to the channel URL with a monotonic id and a send timestamp. We instrument four segments: (S1) producer→relay intake (relay receive minus send); (S2) relay→extension delivery, measured separately for push (SSE) and poll modes; (S3) extension→localhost→extension (the `performance.now()` delta the worker already records as `latencyMs`, rounded to whole milliseconds, which bounds S3 reporting resolution); and (S4) end-to-end (local receive minus producer send). Each configuration is run for the same event count, warmed, and repeated across seeds; we report median and p95 per segment, and the distribution must be stamped with a hardware/region fingerprint (client OS, browser build, relay region, Neon region, git commit). Push and poll are compared at equal event load to isolate the SSE latency benefit. A cold-start variant deliberately idles the MV3 worker until termination, then sends one event, to measure the alarm-driven wake-and-rehydrate penalty. smee-client and ngrok are run on the same harness at equal event load as external baselines, so BridgeHook's numbers are not reported in isolation. Significance between configurations is assessed with bootstrap confidence intervals on the median rather than parametric tests, which the seed counts do not justify.

5.3 Threats to validity

The latency table is unpopulated, so this paper makes no empirical performance claim; the methodology is specified but its results are future work. The feature comparison (Table 2) reflects each tool's *documented* behavior, not behavior we re-measured, and vendor tiers change over time. The planned measurement uses a single client, region, and relay deployment, so results, once collected, will not generalize across regions without repetition. The security analysis reasons about the shipping code path; it is not a formal proof and has not been independently audited.

Table 3: End-to-end latency (ms) — methodology in §5.2. All cells to be filled from real runs.

Configuration	S1 intake med / p95	S2 delivery med / p95	S3 local round-trip med / p95	S4 end-to-end med / p95
BridgeHook push (SSE)	— / —	— / —	— / —	— / —
BridgeHook poll (2 s)	— / —	— / —	— / —	— / —
BridgeHook poll (15 s SSE-healthy)	— / —	— / —	— / —	— / —
BridgeHook cold MV3 wake	— / —	— / —	— / —	— / —
smee-client (baseline)	— / —	— / —	— / —	— / —
ngrok (baseline)	— / —	— / —	— / —	— / —

6 Limitations

Chromium and MV3 only. The shipping extension targets Chromium (Manifest V3, module service worker); Firefox is not yet supported. The zero-install claim is relative to having a Chromium browser.

Service-worker and tab lifecycle. The MV3 worker can be terminated at any time; the heartbeat is bounded below by the 30 s alarm minimum, so a poll-only configuration can lag a webhook by up to one heartbeat plus rehydrate cost after a cold wake. Push mode masks this while the SSE stream is healthy, but push requires the dashboard session cookie and so is unavailable to device-token and self-host configurations.

Trust in the relay operator. The relay sees every payload in plaintext (as do smee.io and any hosted relay). Signing protects integrity — a hijacked or malicious relay cannot forge signed claims or responses — but does not provide payload confidentiality against the operator. Self-hosting the relay is the mitigation for users who cannot accept that exposure.

Private Network Access for extensions. The capability the design rests on — an extension reaching localhost — could be narrowed if PNA-style rules are extended to extension contexts [15, 14]. This is a roadmap risk we do not control; the desktop (Tauri) bridge is the planned fallback if the browser path is foreclosed.

Single-region relay and free-tier caps. The relay is a single logical deployment; cross-region producers pay the intake hop to that region. The hosted free tier enforces a daily event cap (surfaced as `402 quota`) and the Workers 10 ms CPU limit shaped the choice of PBKDF2 over a heavier KDF.

Custom domain not provisioned. `bridgehook.dev` is not yet registered; the tool runs on `*.workers.dev` / `*.pages.dev`.

7 Related Work

Reverse tunnels [10, 2, 7, 11] publish a local port directly and are general-purpose (any TCP/HTTP service), but require a local agent. Relay-plus-SSE tools [13, 12, 1] are the direct antecedents of BridgeHook's transport; gosmee additionally lets operators self-host the relay, which BridgeHook also supports. Production webhook platforms [17, 6] add durable delivery, retries, and permanent URLs, with CLIs that forward to localhost for development; Hookdeck in particular offers permanent free URLs and preserved history, overlapping BridgeHook's stable-URL goal. The closest antecedent is hooks-localhost [8], which also makes the browser the forwarding agent, but does so from a page over WebSocket and so inherits the CORS-allowlist and open-tab constraints of §2; running the agent as an extension is what removes them. Across these tools the forwarding agent is either a local

process or a CORS-constrained tab. BridgeHook's distinct position is no-local-process forwarding via an extension agent, combined with the Durable Object rendezvous and untrusted-relay signing described above.

8 Conclusion

BridgeHook removes the install and the local daemon from webhook local-testing by using a browser extension as the forwarding agent, which is the one client form that can legally reach `http://localhost` from an HTTPS-relayed flow. The relay+SSE transport is inherited from smee.io and gosmee and credited as such; the engineering contribution is the surrounding system enumerated in §1, and in particular the choice to run the forwarding agent as an extension so it clears the browser's localhost-reach restrictions without a local process. The system runs today on Cloudflare free-tier hosts. The remaining work is empirical: the latency methodology of §5.2 must be run to populate Table 3, and the desktop bridge and custom-domain deployment are designed but not yet shipped.

Artifact availability

The implementation is open source at `https://github.com/hallelx2/bridgehook` (commit `83c10d3` at the time of writing), released under the MIT License. The repository contains the relay Worker, the Manifest V3 extension, the web dashboard, and the shared schema. The relay self-hosts on Cloudflare Workers: setting `BETTER_AUTH_SECRET` enables hosted multi-tenant mode, and leaving it unset runs a single-user self-host relay with no auth routes. The latency harness of §5.2 will be released under `paper/eval/` alongside its result files.

References

[1] C. Boudjnah. gosmee — command line server and client for webhooks deliveries. `https://github.com/chmouel/gosmee`, 2024. Go implementation of a smee-style relay (server) and forwarding client; supports self-hosting the relay and replaying saved payloads.

[2] Cloudflare. Cloudflare tunnel (`cloudflared`). `https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/`, 2026. Runs the `cloudflared` daemon locally to establish an outbound-only tunnel exposing local services through Cloudflare's network.

[3] Cloudflare. What are durable objects? Cloudflare Durable Objects documentation, 2026. A Durable Object is a single addressable instance combining compute and storage; all requests for a given ID route to the same instance, enabling coordinated in-memory state.

[4] GitHub. About webhooks. GitHub Docs, 2026. GitHub delivers event payloads via HTTP POST to a configured URL, with HMAC signatures over the body.

[5] Google Chrome. The extension service worker lifecycle. Chrome for Developers — Extensions documentation, 2026. Manifest V3 background logic runs in an event-driven service worker that terminates on inactivity; event listeners reset a 30-second idle timer.

[6] Hookdeck. Hookdeck cli — forward webhooks to localhost. https://github.com/hookdeck/hookdeck-cli, 2026. hookdeck listen creates a permanent public URL that tunnels events to localhost through Hookdeck's event gateway, preserving event history across sessions.

[7] localtunnel. localtunnel — expose your localhost to the world. https://github.com/localtunnel/localtunnel, 2024. Node client (lt) that tunnels a local port to a public *.localtunnel.me URL via a tunnel server.

[8] M. Loewenstein. hooks-localhost — forward webhooks to your localhost in the browser. https://github.com/MoritzLoewenstein/hooks-localhost, 2023. A relay that broadcasts received webhooks over WebSocket to an open browser tab, where page JavaScript re-issues the request to localhost; relies on the local server's CORS configuration and a kept-open tab.

[9] MDN Web Docs. Mixed content. MDN Web Docs, 2026. Browsers block or upgrade HTTP subresources requested from an HTTPS page; an HTTPS page cannot issue a plain-HTTP fetch.

[10] ngrok. ngrok — unified ingress platform / secure tunnels to localhost. https://ngrok.com/docs, 2026. Reverse-tunnel agent: a local daemon opens an outbound connection and exposes a public URL bound to a local port.

[11] Pinggy. Pinggy — simple localhost tunnels over ssh. https://pinggy.io/docs/, 2026. Zero-install tunnel using the system SSH client and remote port forwarding (ssh -p 443 -R0:localhost:PORT free.pinggy.io); free tunnels time out after 60 minutes.

[12] Probot. smee-client — receives payloads then sends them to your local server. https://github.com/probot/smee-client, 2024. Node CLI client that subscribes to a smee.io channel over SSE and re-POSTs payloads to a local URL.

[13] Probot. smee.io — webhook payload delivery service. https://github.com/probot/smee.io, 2024. Open-source webhook payload delivery service using Server-Sent Events; channels are unauthenticated and payloads are not stored. Accessed.

[14] T. Rigoudy. Private network access: introducing preflights. Chrome for Developers Blog, 2022. Chrome sends a CORS preflight (Access-Control-Request-Private-Network: true) ahead of public-to-private subresource requests; the target must answer with Access-Control-Allow-Private-Network: true.

[15] T. Rigoudy and M. West. Private network access. Draft Community Group Report, Web Platform Incubator Community Group (WICG), 2024. Specifies Fetch/HTML modifications restricting requests from public contexts to private/local network endpoints; formerly CORS-RFC1918.

[16] Stripe. Receive stripe events in your webhook endpoint. Stripe API documentation, 2026. Stripe POSTs signed event payloads to a developer-configured HTTPS endpoint; signatures are verified with a per-endpoint secret.

[17] Svix. svix-cli — a cli for the svix webhooks service (svix listen). https://github.com/svix/svix-webhooks/tree/main/svix-cli, 2026. The listen command acts as a proxy forwarding inbound requests to a local URL; the broader Svix server is an open-source (MIT) webhook-sending service.

[18] Webhook Relay. Webhook relay — receive webhooks on localhost. `https://webhookrelay.com/`, 2026. Commercial webhook forwarding service; an agent (CLI or container) subscribes to a hosted bucket and relays events to a local endpoint.

[19] WHATWG. Html living standard — server-sent events. WHATWG HTML Standard, § 9.2, 2026. Defines the `text/event-stream` format and the `EventSource` interface for server-pushed events over a long-lived HTTP response.